GRAPH-AUGMENTED MADDPG FOR SCALABLE MULTI-AGENT REINFORCEMENT LEARNING

Tristan Thakur, Bianca Jayaraman, & Richard Hayden Narey

1 INTRODUCTION

Large-scale multi-agent systems pose significant challenges for policy learning due to the massive joint state-action spaces and the complex, shifting relationships between entities, especially in mixed cooperative-competitive environments. A fundamental obstacle in this domain is non-stationarity, where multiple agents simultaneously update their behaviors, effectively changing the environment's dynamics from the perspective of any single agent and violating the Markov assumptions required for traditional reinforcement learning. Rule-based methods often fail to capture these dynamic interactions, making the modeling of inter-agent coordination a significant research issue.

We address these challenges by employing graph neural networks to provide a relational inductive bias, enabling agents to reason about their spatial and intent-based connections through a structured message-passing paradigm. By integrating attention mechanisms, we allow agents to selectively filter communication and prioritize relevant local interactions, which mitigates the noise and computational overhead typically found in fully connected systems. We conduct our investigation within the predator-prey framework, utilizing a modular approach that abstracts the underlying centralized training, decentralized execution logic to isolate and evaluate the impact of different graph architectures. Agents take local, pre-encoded observation vectors as inputs and output an action that represents its movement in the environment. This research is highly significant for the field of swarm robotics, as it develops scalable and permutation-invariant coordination strategies that allow fleets of autonomous systems to generalize their cooperative behaviors to varying population sizes and dense, cluttered environments.

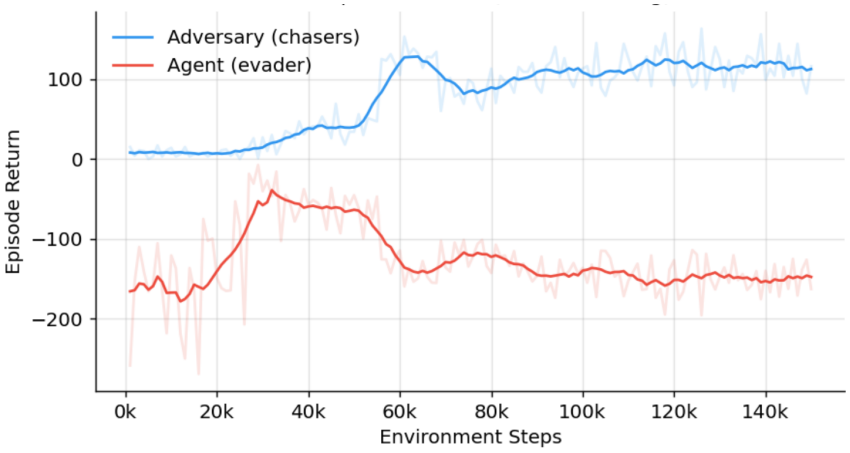

Figure 1: Baseline MADDPG: Smoothed Train Episode Return

2 RELATED WORK

Previous work in MARL has exemplified why non-stationarity is a major obstacle, since agents simultaneously updating themselves without regard for each other can change the state significantly. The Centralized Training with Decentralized Execution (CTDE) paradigm has been proposed to mitigate this, which allows agents to train with full information of the environment but not test with it. Researchers have also proposed methods like Multi-Agent Deep Deterministic Policy Gradient

(MADDPG) which uses centralized critics to stabilize learning in environments like ours Lowe et al. (2017). Other work has focused on value estimation using factorization, such as QMIX, which breaks a joint value function down into agent-wise components Rashid et al. (2018). However, this work does not consider agents without full information, so its usefulness for our goals remains to be seen.

Other work that does surrounding agents with discrete information has focused on inter-agent communication. This typically relies on fully connected message passing, but this is not an unreasonable assumption in our context. CommNet and DIAL demonstrate the ability for agents to learn differentiable communication protocols, which allows this communication to be built into the learning Das et al. (2020); Rashid et al. (2018). This does raise the issue of information dilution at larger numbers of agents. This issue could be resolved either at the sender or recipient end. On the sender end, TarMAC has been proposed which uses social attention to allow agents to target only the relevant recipients for a message [5]. On the receiver side, graph-based techniques to represent agents allow for implicit bias in information sending to reduce dilution. Deep Graph Networks (DGNs) for message passing have shown the ability to generalize to a varying number of agents Liu et al. (2019). This is implemented for MARL with G2ANet, which can weigh or even cull agent interactions to emphasize information sharing between relevant neighbors only Veličković et al. (2018).

These works and other prior works typically rely on state encoders or value factorization to assign rewards. In contrast, our proposed Communication-Abstraction pattern explicitly models the dynamic relationships between agents. This has not previously been combined with a GNN abstraction for communication, as many works assume fully connected message passing. Combining these techniques will help reduce noise and make our model more invariant to the number of agents.

3 Problem Decomposition & Detailed Technical Approach

In this section, we formalize our proposed methodology, which integrates Graph Neural Networks (GNNs) with MADDPG to address the coordination challenges in large-scale multi-agent systems. We also state three hypotheses about our anticipated results.

3.1 Technical Approach

We implement a MADDPG pipeline in the PettingZoo simple_tag_v3 predator-prey environment with centralized training and decentralized execution. The environment is configured to have three chasers (the adversaries), one agent, two obstacles, and 100 steps per episode. However, it is capable of handling more than one agent. For each agent, we construct a decentralized actor that converts that agent's observation to an action. We also train a centralized critic that has access to the observations and actions of all agents. The critic builds a global representation and feeds it into a multi-agent MLP to score the state-action pairs pertaining to each agent. This allows us to maintain the MADDPG benefit of stabilized learning with centralized information, but the agents still maintain their local scope to maintain realism. We chose this environment because it is a well-studied benchmark for multi agent coordination, and is simple enough to clearly observe agent behavior and compare methods. It has both cooperative and competitive dynamics due to the varying numbers of predators and prey, which allows us to view both sides of agent learning rather than just focusing on one.

The training loop uses the standard off-policy deep reinforcement learning approach. Experience is gathered through a SyncDataCollector, and actions are produced by composing each actor with gaussian noise using AdditiveGaussianModule. Transitions are stored in a shared replay buffer. During optimization, we sample mini-batches from replay and perform critic and actor updates using DDPGLoss. Target networks are updated with Polyak averaging, and gradients are clipped to maintain stability. The evaders can optionally be frozen after a certain number of iterations to let training focus on the coordinated behavior of the chasers.

This main architecture is consistent across both our baseline and graph-augmented extension. This extension will sit between the raw observation input and the actor/critic networks. Instead of treating each agent's observation as a simple vector, we will represent the multi-agent state as a graph whose nodes correspond to agents and edges encode the relationships between agents (like distance). A graph neural network layer can process this information into a relational embedding for each agent

which can be used in action selection for the agent and value estimation for the critic. The remainder of the MADDPG training process remains the same to allow our baseline and experiment to be compared effectively. Thus, any performance difference between our baseline and experimental results will be attributable to the graph augmentation we introduce.

To better test our graph augmentation method, this architecture (both for baseline and experimental) somewhat deviates from typical "standard practice" MADDPG implementation. Specifically, typical implementations perform feature extraction within the actor/critic networks. This means that it is difficult to separate observation encoding from policy decisions. In contrast, our architecture separates out the feature encoding into a module that then sends it to both the actors and critic. This unifies the representation being seen by the critic and actors, which allows greater insight (both for us and the critic) on adversary policy strength.

This design choice was made with our graph augmentation in mind. Since we don't compute observation representations in separate MLPs, we can easily slot in our GNN for the MLP encoder without having to restructure the architecture. This allows us to match the baseline and experimental architectures as closely as possible to strengthen the validity of our results. Currently, the graph-based encoder is in the process of being integrated into our training pipeline.

3.2 Evaluation Procedure

We evaluated our baseline MADDPG implementation using the PettingZoo simple_tag_v3 predator-prey environment. We configured this environment to have 3 adversarial chasers, 1 evading agent, and 2 stationary obstacles with 100 steps per episode. All of these agents were trained using the Centralized Training for Decentralized Execution (CTDE) paradigm for 150,000 total environment steps Amato (2024). We evaluated this performance using a deterministic policy at intermittent checkpoints throughout training. High variance is typical during MARL training, thus we evaluated inter-quartile mean along with 95% confidence intervals averaged across 5 random seeds. The main metrics we used for both chasers and evading agents are episode return. Using both of these returns, we were able to capture how well the chasers cooperate together and how well the evaders can evade them.

3.3 Hypotheses

Based on our approach, we have formulated the following hypotheses:

- **H1:** Graph augmented agents will achieve a higher final episode return than the MLP baseline due to relational embeddings providing further context between agents.
- **H2:** Graph-augmented agents will reach higher levels of competitive performance in fewer environment steps when compared to our baseline. We hypothesize the structural relationships from the graph model will reduce the experience needed to discover chase strategies.
- **H3:** Graph-augmented agents will output results with lower variance in final performance across runs with different random seeds compared to the MLP baseline due to the graph representation allowing for more stability in learning of coordinated agent behavior.

4 Experiments & Results

4.1 Dataset

We conducted all experiments in the Petting Zoo simple_tag_v3 predator-prey environment which is a standard benchmark for multi-agent RL research. This environment provides both continuous state and action spaces. In other words, each agent is able observe the position and velocity of itself and other agents. Each episode consists of 100 time steps and rewards are formulated to both incentivize chasers to tag evaders and penalize evaders for being caught. All of the training data we used is generated locally through environment interaction. We collected 150,000 environment steps per run through the SyncDataCollector, which is then stored in a shared replay buffer.

4.2 Results

We completed five full baseline runs of the base MADDPG implementation. In all runs, agents were able to learn how to coordinate in the simple_tag_v3 environment. However, there was a lot of variability in performance across runs.

Group	Low	High	Mean	Std
Adversaries	49.0	484.0	279.4	172.1
Agent	-738.4	-406.2	-545.5	136.6

Table 1: Summary statistics of final evaluation returns across five baseline MADDPG runs.

These results indicate that some of the runs were able to learn effective policies very well, but other runs stalled out with less-than-great results. This indicates that the baseline MADDPG implementation is more sensitive to randomness generated from the initialization and exploration steps. This indicates that the baseline is capable of learning strong policies, but it is far from guaranteed.

The agent performance has the opposite relationship, which is expected for this environment. In runs where the adversaries learned strong policies, the agent struggled more. On the other hand, on the weaker adversarial runs, the agent was able to evade them more effectively. This inverse relationship confirms the core assumption for our evaluation metric: better adversarial teamwork negatively impacts the ability for the agent to evade them.

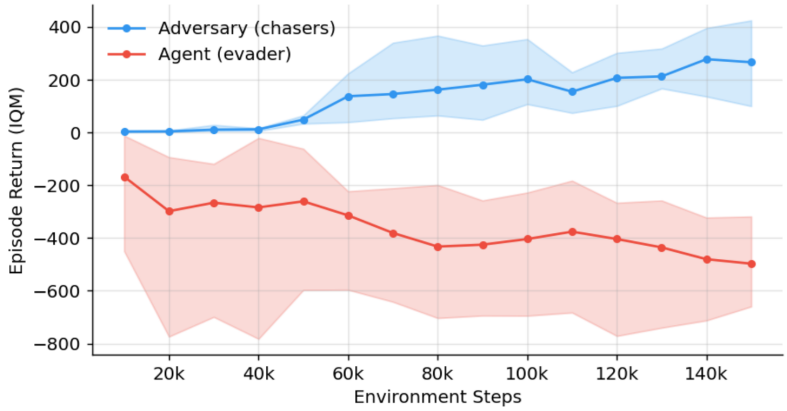

Figure 2: Baseline Eval Episode Return - IQM $\pm$ 95% CI (All Runs)

Overall, these results indicate that our baseline MADDPG can learn coordinated behavior, which is reflected by strong adversarial performance and weak agent performance. However, the variations between runs indicate that it is sensitive to small perturbations in state, which is not ideal. Our graph-augmented MADDPG should be able to help reduce this sensitivity and achieve stronger results overall.

5 Next Steps

We intentionally structured our MADDPG implementation to enable our graph-based representation. Because the observation encoding stage is already isolated and separate from the actor/critic components, we can easily replace it with a graph neural network without further restructuring. Agent observations will be structured as nodes within this graph, and edges will represent information pertaining to the inter-agent relationships, namely distance and separating obstacles. The resultant embeddings can then be sent directly to the actor/critic components without modifying any other parts of the training process. This restructuring concern was a problem we identified early on, which allowed us to avoid it from the start.

The main challenge we anticipate looking forward is finding a balance between encoding useful information in our graph structure and encoding so much information that the graph becomes too

complex to allow improved learning and performance. There will always be some relation between adversaries, but a fully connected graph would likely introduce too much noise and hurt learning. On the other hand, an overly simplified graph may not contain enough information to actually benefit the adversaries and allow for strong coordination. To address this concern, we will begin with a simpler graph implementation that focuses on pure proximity between agents which allows edges to be easily culled based on distance. We can continue to add to this, adding considerations for factors like obstacles, until we have a strong graph representation that doesn't introduce too much noise.

Our main success criteria for the final deliverables is a fully functional graph representation that is tuned for the simple_tag_v3 environment, but remains scalable to different parameters including number of adversaries and agents. With this augmentation, we will be able to make strong, evidence-based comparisons between the baseline MADDPG and our GNN-augmented MADDPG.

5.1 Timeline

The first step in our timeline is to get a GNN component working within our model. This should be achievable within 1-2 weeks, since our architecture was designed with it in mind. After that, we will have an additional 1-2 weeks to tune the graph parameters to address the challenges mentioned above. The final weeks of the timeline can then be spent running full runs of our experimental model to generate a strong data corpus that we can compare to the baseline to fully measure the effectiveness of our proposed method.

References

Christopher Amato. An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning, 2024. URL https://arxiv.org/abs/2409.03052.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. Tarmac: Targeted multi-agent communication, 2020. URL https://arxiv.org/abs/1810.11187.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network, 2019. URL https://arxiv.org/abs/1911.10715.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *CoRR*, abs/1706.02275, 2017. URL http://arxiv.org/abs/1706.02275.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning, 2018. URL https://arxiv.org/abs/1803.11485.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018. URL https://arxiv.org/abs/1710.10903.